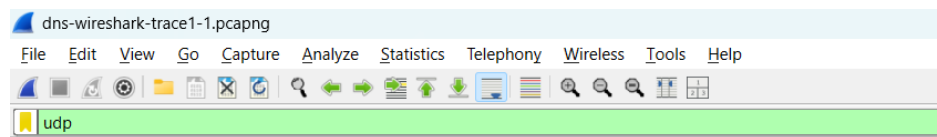


Assignment #4 - Wireshark UDP & TCP

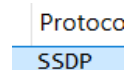
UDP Portion

Question 1: Select the first UDP segment in your trace. What is the packet number of this segment in the trace file? What type of application-layer payload or protocol message is being carried in this UDP segment? Look at the details of this packet in Wireshark. How many fields are there in the UDP header? What are the names of these fields?

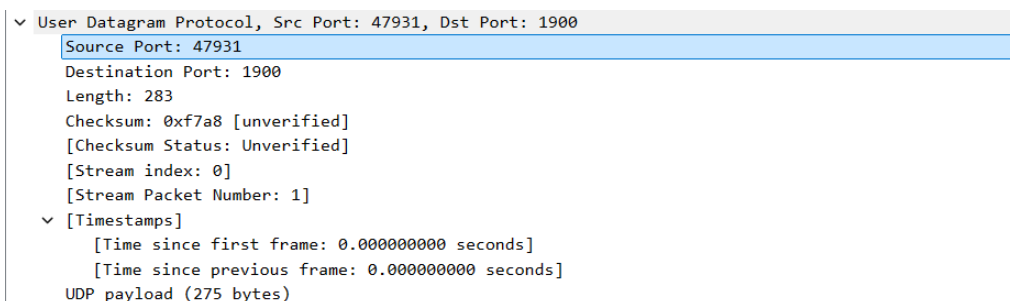
- Packet #: Packet #5



- Application-Layer Payload / Protocol Message Type: **SSDP**



- Number of Fields in Header:
 - 4
- Names of Fields in Header:
 - Source Port, Destination Port, Checksum, Length



Question 2: By consulting the displayed information in Wireshark's packet content field for this packet, what is the length in bytes of each of the UDP header fields?

- Source Port: 2 bytes
- Destination Port: 2 bytes
- Checksum: 2 bytes
- Length: 2 bytes

The image shows a Wireshark packet capture. The left pane shows the packet list with 'Frame 5: Packet, 317 bytes on wire (2536 bits), 317 bytes captured (2536 bits) on interface en0, id 0'. The middle pane shows the packet details for 'User Datagram Protocol, Src Port: 47931, Dst Port: 1900'. The right pane shows the packet bytes in hexadecimal and ASCII. The packet is an HTTP 1.1 POST request to 'http://10.0.0.2:5449152 /ups_dev'. The packet length is 317 bytes, which includes the Ethernet II header (14 bytes), Internet Protocol Version 4 header (20 bytes), and the User Datagram Protocol header (8 bytes) plus the payload (275 bytes).

Question 3: The value in the length field is the length of what? Verify your claim with your captured UDP packet.

- The value in the length field is the length of the entire packet (payload + header)
- 275 (payload) + 8 (header) = 283

The image shows the 'User Datagram Protocol' details in Wireshark. The fields are: Source Port: 47931, Destination Port: 1900, Length: 283, Checksum: 0xf7a8 [unverified], [Checksum Status: Unverified], [Stream index: 0], [Stream Packet Number: 1], [Timestamps], and UDP payload (275 bytes).

Question 4: What is the maximum number of bytes that can be included in a UDP payload?

- 65527
- Max Length of Entire Packet: 65535 ($2^{16} - 1$)

- Header Size: 8 bytes
- $65535 - 8 = 65527$

Question 5: What is the largest possible source port number?

- **65535**
- $2^{16} = 65535$

Question 6: What is the protocol number for UDP? Give your answer in decimal notation. To answer this question, you'll need to look into the Protocol field of the IP datagram containing this UDP segment.

- Protocol Number: 17

```
Protocol: UDP (17)
Header Checksum: 0xd902 [validation disabled]
[Header checksum status: Unverified]
Source Address: 10.0.0.254
```

Question 7: Examine the pair of UDP packets in which your host sends the first UDP packet and the second UDP packet is a reply to this first UDP packet. (Hint: for a second packet to be sent in response to a first packet, the sender of the first packet should be the destination of the second packet). What is the packet number of the first of these two UDP segments in the trace file? What is the value in the source port field in this UDP segment? What is the packet number of the second of these two UDP segments in the trace file? What is the value in the source port field in this second UDP segment? What is the value in the destination port field in this second UDP segment? Describe the relationship between the two port numbers in the two packets.

- Packet # of the First: **15**
- Source Port # of the First: **58350**
- Packet # of the Second: **17**
- Source Port # of the Second: **53**
- Destination Port # of Second: **58350**
- Relationship: Port # 58350 is the source port sending a DNS query to port 53, which is commonly used for DNS. These two packets are

communicating about a DNS query. Port 53 sends back the relevant information to answer the query back to the port 58350.

15	15:34:40.807265	10.0.0.44	75.75.75.75	DNS	77 Standard query 0x3c29 A gaia.cs.umass.edu
17	15:34:40.831173	75.75.75.75	10.0.0.44	DNS	93 Standard query response 0x3c29 A gaia.cs.umass.edu A 128.119.245.12
30	15:34:40.909593	10.0.0.44	75.75.75.75	DNS	83 Standard query 0xeda4 A maxcdn.bootstrapcdn.com
31	15:34:40.910715	10.0.0.44	75.75.75.75	DNS	79 Standard query 0xa79 A ajax.googleapis.com
35	15:34:40.927250	75.75.75.75	10.0.0.44	DNS	135 Standard query response 0xeda4 A maxcdn.bootstrapcdn.com CNAME cds.j3z9t3p6.hucdn.net A 209.197.3.15
36	15:34:40.931107	75.75.75.75	10.0.0.44	DNS	95 Standard query response 0xa79 A ajax.googleapis.com A 172.217.12.202
521	15:34:41.160429	10.0.0.44	75.75.75.75	DNS	75 Standard query 0xdcfa A www.pearson.com
522	15:34:41.160594	10.0.0.44	75.75.75.75	DNS	79 Standard query 0xb436 A www.vitalsource.com
523	15:34:41.160799	10.0.0.44	75.75.75.75	DNS	72 Standard query 0xd3a3 A redshelf.com

> Frame 15: Packet, 77 bytes on wire (616 bits), 77 bytes captured (616 bits) on interface en0, id 0	0000	00 50 f1 80 00 00 78 4f 43 98 d9 27 08 00 45 00	P.....xO C.....E..
> Ethernet II, Src: Apple_98:d9:27 (78:4f:43:98:d9:27), Dst: Maxlinear_80:00:00 (00:50:f1:80:00:00)	0010	00 3f 4c 16 00 00 ff 11 ce d5 0a 00 00 2c 4b 4b	..?L.....KK
> Internet Protocol Version 4, Src: 10.0.0.44, Dst: 75.75.75.75	0020	4b 4b e3 ee 00 35 00 2b c3 1d 3c 29 01 00 00 01	KK...S+...<)....
> User Datagram Protocol, Src Port: 58350, Dst Port: 53	0030	00 00 00 00 00 00 04 67 61 69 61 02 63 73 05 75	g aia:cs-u
> Source Port: 58350	0040	6d 61 73 73 03 65 64 75 00 00 01 00 01	mass-edu
> Destination Port: 53			
> Length: 43			
> Checksum: 0xc31d [unverified]			
> [Checksum Status: Unverified]			
> [Stream index: 1]			
> [Stream Packet Number: 1]			
> [Timestamps]			
> UDP payload (35 bytes)			
> Domain Name System (query)			

TCP Portion

Question 1: What is the IP address and TCP port number used by the client computer (source) that is transferring the alice.txt file to gaia.cs.umass.edu? To answer this question, it's probably easiest to select an HTTP message and explore the details of the TCP packet used to carry this HTTP message, using the "details of the selected packet header window".

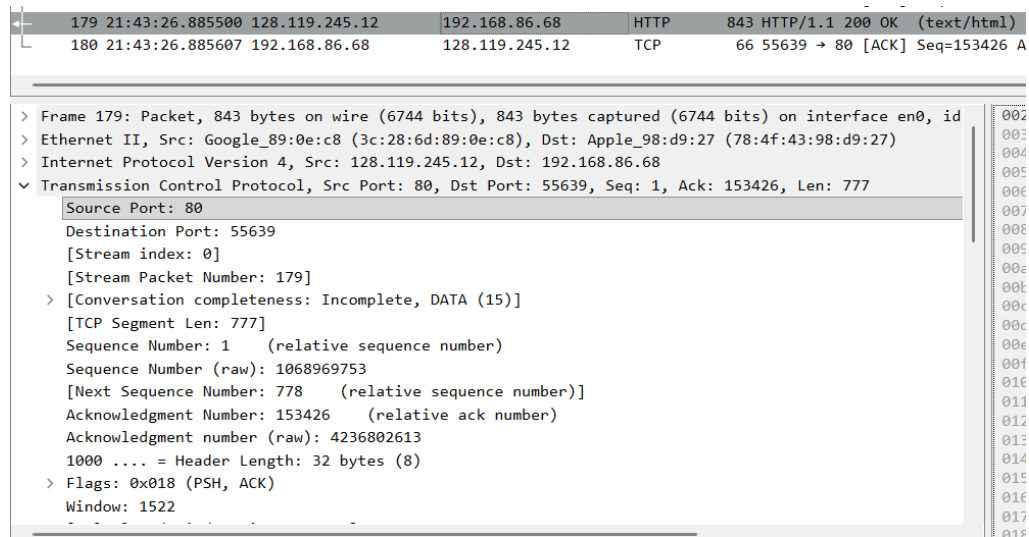
- Client IP: 192.168.86.68
- Client Port Number: 55639

```
> Internet Protocol Version 4, Src: 192.168.86.68, Dst: 128.119.245.12
> Transmission Control Protocol, Src Port: 55639, Dst Port: 80, Seq: 152041, Ack: 1, Len: 1385
> [106 Reassembled TCP Segments (153425 bytes): #4(1448), #5(1448), #6(1448), #9(1448), #10(1448), #11(
  > Hypertext Transfer Protocol
    > POST /wireshark-labs/lab3-1-reply.htm HTTP/1.1\r\n
      Host: gaia.cs.umass.edu\r\n
      User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:85.0) Gecko/20100101 Firefox/85.0\r\n
      Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8\r\n
      Accept-Language: en-US,en;q=0.5\r\n
      Accept-Encoding: gzip, deflate\r\n
      Content-Type: multipart/form-data; boundary=-----193950769437984536211154302:
    > Content-Length: 152359\r\n
      Origin: http://gaia.cs.umass.edu\r\n
      DNT: 1\r\n
      Connection: keep-alive\r\n
      Referer: http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html\r\n
```

Question 2: What is the IP address of gaia.cs.umass.edu? On what port number is it sending and receiving TCP segments for this connection?

- IP Address: 128.119.245.12

- Port Number: 80



Question 3: What is the sequence number of the TCP SYN segment that is used to initiate the TCP connection between the client computer and gaia.cs.umass.edu? (Note: this is the “raw” sequence number carried in the TCP segment itself; it is NOT the packet # in the “No.” column in the Wireshark window. Remember that there is no such thing as a “packet number” in TCP or UDP, as you know, there are sequence numbers in TCP and that’s what we’re after here. Also note that this is not the relative sequence number with respect to the starting sequence number of this TCP session.) What is it in this TCP segment that identifies the segment as a SYN segment? Will the TCP receiver in this session be able to use Selective Acknowledgements (allowing TCP to function a bit more like a “selective repeat” receiver)?

- Sequence #: 4236649187, used to initiate the three way handshake
- How it is identified as a SYN Segment: The SYN flag is set (0x002)

```
Sequence Number (raw): 4236649187
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1011 .... = Header Length: 44 bytes (11)
> Flags: 0x002 (SYN)
Window: 65535
[Calculated window size: 65535]
Checksum: 0xa1e4 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
```

- Will the TCP receiver in this session be able to use selective acknowledgements: Yes

), Timestamps, SACK permitted, End of Option List (l

Question 4: What is the sequence number of the SYNACK segment sent by gaia.cs.umass.edu to the client computer in reply to the SYN? What is it in the segment that identifies the segment as a SYNACK segment? What is the value of the Acknowledgement field in the SYNACK segment? How did gaia.cs.umass.edu determine that value?

- Sequence Number: 1068969752
- What is in the segment to identify it as SYNACK: The SYNACK flag is set (0x012)
- Value of Acknowledgement Field: 4236649188

```
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 1068969752
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 4236649188
1010 .... = Header Length: 40 bytes (10)
> Flags: 0x012 (SYN, ACK)
Window: 28960
[Calculated window size: 28960]
Checksum: 0x47b4 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
> Options: (20 bytes), Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP), Window :
> [Timestamps]
```

- How it determined that value: It added 1 to the Sequence number that it received most recently. This is to indicate that it received that SYN packet and that the next packet it expects should have a Sequence number of 4236649188

Question 5: What is the sequence number of the TCP segment containing the header of the HTTP POST command? Note that in order to find the POST message header, you'll need to dig into the packet content field at the bottom of the

Wireshark window, looking for a segment with the ASCII text “POST” within its DATA field. How many bytes of data are contained in the payload (data) field of this TCP segment? Did all of the data in the transferred file alice.txt fit into this single segment?

- Sequence # of TCP segment containing POST command: 4236649188

4	21:43:26.716922	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=1449 Ack=1 Win=131712 Len=1448 TSval=725607532 TSecr=3913851370 [TCP PDU reassembled in 153]
5	21:43:26.716923	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=1449 Ack=1 Win=131712 Len=1448 TSval=725607532 TSecr=3913851370 [TCP PDU reassembled in 153]
6	21:43:26.716924	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=2897 Ack=1 Win=131712 Len=1448 TSval=725607532 TSecr=3913851370 [TCP PDU reassembled in 153]
7	21:43:26.745546	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=1449 Win=31872 Len=0 TSval=3913851399 TSecr=725607532
8	21:43:26.745551	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=2897 Win=34816 Len=0 TSval=3913851400 TSecr=725607532
9	21:43:26.745649	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=4345 Ack=1 Win=131712 Len=1448 TSval=725607560 TSecr=3913851399 [TCP PDU reassembled in 153]
10	21:43:26.745650	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=5793 Ack=1 Win=131712 Len=1448 TSval=725607560 TSecr=3913851399 [TCP PDU reassembled in 153]
11	21:43:26.745729	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=7241 Ack=1 Win=131712 Len=1448 TSval=725607560 TSecr=3913851400 [TCP PDU reassembled in 153]
12	21:43:26.745730	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=8689 Ack=1 Win=131712 Len=1448 TSval=725607560 TSecr=3913851400 [TCP PDU reassembled in 153]
13	21:43:26.746501	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=4345 Win=37760 Len=0 TSval=3913851400 TSecr=725607532
14	21:43:26.746585	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=10137 Ack=1 Win=131712 Len=1448 TSval=725607560 TSecr=3913851400 [TCP PDU reassembled in 153]
15	21:43:26.746586	192.168.86.68	128.119.245.12	TCP	1514 → 55639 → 80 [ACK] Seq=11585 Ack=1 Win=131712 Len=1448 TSval=725607560 TSecr=3913851400 [TCP PDU reassembled in 153]
16	21:43:26.773643	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=5793 Win=40576 Len=0 TSval=3913851421 TSecr=725607560
17	21:43:26.773646	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=7241 Win=43520 Len=0 TSval=3913851422 TSecr=725607560

[Completeness Flags: -DASS] [TCP Segment Len: 1448] Sequence Number: 1 (relative sequence number) Sequence Number (raw): 4236649188 [Next Sequence Number: 1449 (relative sequence number)] Acknowledgment Number: 1 (relative ack number) Acknowledgment Number (raw): 1068969753 1000 ... = Header Length: 32 bytes (8) > Flags: 0x010 (ACK) Window: 2058 [Calculated window size: 131712] [Window size scaling factor: 64] Checksum: 0xb2d1 [unverified] [Checksum Status: Unverified] Urgent Pointer: 0 > Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps > [Timestamps] > [SEQ/ACK analysis]	0040 a1 ea 80 4f 53 54 20 2f 77 69 72 65 73 68 61 72 POST / wireshar 0050 6b 2d 6c 61 62 73 2f 6c 61 62 33 2d 31 2d 72 65 k-labs/l ab3-1-re 0060 70 6c 79 2e 68 74 6d 20 48 54 54 50 2f 31 2e 31 ply.htm HTTP/1.1 0070 8d 0a 48 6f 73 74 3a 20 67 61 60 61 2e 63 73 2a r-Huati gale.ca 0080 75 6d 61 73 73 2e 65 64 75 0d 0a 55 73 65 72 2d umass.ed u-User- 0090 41 67 65 6e 74 3a 20 4d 6f 7a 69 6c 6c 61 2f 35 Agent: Mozilla/5 00a0 2e 30 20 28 4d 61 63 69 6e 74 6f 73 68 3b 20 49 i0 (Maci ntosh; I 00b0 6e 74 65 6c 20 4d 61 63 20 4f 53 20 38 20 31 30 tel Mac OS X 10 00c0 2e 31 35 3b 20 72 76 3a 38 35 2e 30 29 20 47 65 ;5; rv:85.0) Ge 00d0 63 6b 6f 2f 32 30 31 30 30 31 30 31 20 46 69 72 cko/2010 0101 Fir 00e0 65 66 6f 70 2f 38 35 2e 20 0d 0a 41 63 63 65 70 efox/85.0-+Accep 00f0 74 3a 20 74 65 78 74 2f 68 74 6d 6c 2c 61 70 70 r: text/html,app 0100 6c 69 63 61 74 69 6f 6e 2f 78 68 74 6d 6c 2b 78 lication /xhtml+ 0110 6d 6c 2c 61 70 70 6c 69 63 61 74 69 6f 6e 2f 78 ml,appli cation/x 0120 6d 6c 2b 71 2d 30 2e 39 2c 60 6d 61 67 65 2f 77 l;js=9.9 image/v 0130 65 62 70 2c 2a 2f 2a 3b 71 3d 30 2e 38 8d 8a 41 ebb;/?; c=8.8-+A 0140 63 63 65 70 74 2d 4c 61 6e 67 75 61 67 65 3a 20 ccept-La nguage: 0150 65 6e 2d 55 53 2c 65 6e 3b 71 3d 30 2e 35 8d 8a en-US,en ;q=0.5- 0160 41 63 63 65 70 74 2d 45 6e 63 6f 64 69 6e 67 3a Accept-E ncoding: 0170 20 67 7a 69 70 2c 20 64 65 6e 6c 61 74 65 8d 8a gzip, d eflate- 0180 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 6d 75 Content- Type: mu 0190 6c 74 69 70 61 72 74 2f 66 6f 72 6d 2d 64 61 74 itipart/ form-dat 01a0 61 3b 20 62 6f 75 6e 64 61 72 79 3d 2d 2d 2d 2d er bound ary=---
--	---

- Number of bytes of data in the payload field of the TCP segment: 1448 bytes

```

> [SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (1448 bytes)
[Reassembled PDU in frame: 153]
TCP segment data (1448 bytes)

```

- Did all of the data in the transferred file alice.txt fit into this single segment?
No.

Question 6: Consider the TCP segment containing the HTTP “POST” as the first segment in the data transfer part of the TCP connection.

- At what time was the first segment (the one containing the HTTP POST) in the data-transfer part of the TCP connection sent?
 - 21:43:26.716922

- At what time was the ACK for this first data-containing segment received?
 - 21:43:26.745546
- What is the RTT for this first data-containing segment?
 - 28.624ms
- What is the RTT value of the second-data carrying TCP segment and its ACK?
 - 21:43:26.745551 - 21:43:26.716923 = 28.628ms
- What is the EstimatedRTT value after the ACK for the second data-carrying segment is received? Assume that in making this calculation after the received of the ACK for the second segment, that the initial value of EstimatedRTT is equal to the measured RTT for the first segment, and then is computed using the EstimatedRTT equation, with a value of $\alpha = 0.125$
 - $(1 - 0.125) * 28.624 + 0.125 * 28.628 = 28.6245$
- Note that WireShark has a nice feature that allows you to plot the RTT for each of the TCP segments sent. Select a TCP segment in the "listing of captured packets" window that is being sent from the client to the gaia.cs.umass.edu server. Then select: Statistics → TCP Stream Graph → Round Trip Time Graph

Question 7: What is the length (header plus payload) of each of the first four data-carrying TCP segments?

- #4 → TCP only (**1480**), whole packet (1514)
- #5 → TCP only (**1480**), whole packet (1514)
- #6 → TCP only (**1480**), whole packet (1514)
- #9 → TCP only (**1480**), whole packet (1514)

4	21:43:26.716922	192.168.86.68	128.119.245.12	TCP	1514	55639 → 80 [ACK] Seq=1 Ack=1 Win=131712 Len=1448 TSval=725607532 TSecr=3913851370 [TCP PDU reassembled in 153]
5	21:43:26.716923	192.168.86.68	128.119.245.12	TCP	1514	55639 → 80 [ACK] Seq=1449 Ack=1 Win=131712 Len=1448 TSval=725607532 TSecr=3913851370 [TCP PDU reassembled in 153]
6	21:43:26.716924	192.168.86.68	128.119.245.12	TCP	1514	55639 → 80 [ACK] Seq=2897 Ack=1 Win=131712 Len=1448 TSval=725607532 TSecr=3913851370 [TCP PDU reassembled in 153]
7	21:43:26.745546	128.119.245.12	192.168.86.68	TCP	66	80 → 55639 [ACK] Seq=1 Ack=1449 Win=31872 Len=0 TSval=3913851399 TSecr=725607532
8	21:43:26.745551	128.119.245.12	192.168.86.68	TCP	66	80 → 55639 [ACK] Seq=1 Ack=2897 Win=34816 Len=0 TSval=3913851400 TSecr=725607532
9	21:43:26.745649	192.168.86.68	128.119.245.12	TCP	1514	55639 → 80 [ACK] Seq=4345 Ack=1 Win=131712 Len=1448 TSval=725607560 TSecr=3913851399 [TCP PDU reassembled in 153]

```

v Transmission Control Protocol, Src Port: 55639, Dst Port: 80, Seq: 4345, Ack: 1, Len: 1448
  Source Port: 55639
  Destination Port: 80
  [Stream index: 0]
  [Stream Packet Number: 9]
  > [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 1448]
  Sequence Number: 4345      (relative sequence number)
  Sequence Number (raw): 4236653532
  [Next Sequence Number: 5793      (relative sequence number)]
  Acknowledgment Number: 1    (relative ack number)
  Acknowledgment number (raw): 1068969753
  1000 .... = Header Length: 32 bytes (8)

```

Question 8: What is the minimum amount of available buffer space advertised to the client by gaia.cs.umass.edu among these first four data-carrying TCP segments? Does the lack of receiver buffer space ever throttle the sender for these first four data carrying segments?

- #4 → $2058 \text{ (size)} * 64 \text{ (scaling)} = 131712$
- #5 → $2058 \text{ (size)} * 64 \text{ (scaling)} = 131712$
- #6 → $2058 \text{ (size)} * 64 \text{ (scaling)} = 131712$
- #9 → $2058 \text{ (size)} * 64 \text{ (scaling)} = 131712$
- Minimum: 131712
- The lack of receiver buffer space doesn't throttle the sender for these first four data carrying segments. The window never changes and 131721 is never hit within those first four data carrying segments.

```

v Segment 4345 (raw)
  Window: 2058
  [Calculated window size: 131712]
  [Window size scaling factor: 64]
  Checksum: 0x2fa2 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0

```

- Give the Wireshark reported value for "Window Size Value", which must then be multiplied by the Window Scaling Factor to give the actual number of buffer bytes available at gaia.cs.umass.edu for this connection.

Question 9: Are there any retransmitted segments in the file trace? What did you check for (in the trace) in order to answer this question?

- None

- I looked for any [TCP Retransmission] meta data in the Info section of Wireshark
- You can also look for duplicate SEQ, ACK pairs.

tcp.analysis.retransmission						
No.	Time	Source	Destination	Protocol	Length	Info

Question 10: How much data does the receiver typically acknowledge in an ACK among the first ten data-carrying segments sent from the client to gaia.cs.umass.edu? Can you identify cases where the receiver is ACKing every other received segment among these first ten data-carrying segments?

- #4 → 1514 (1448 TCP)
- #5 → 1514 (1448 TCP)
- #6 → 1514 (1448 TCP)
- #7 → 1448
- #8 → 1448
- #9 → 1514 (1448 TCP)
- #10 → 1514 (1448 TCP)
- #11 → 1514 (1448 TCP)
- #12 → 1514 (1448)
- #13 → 1448
- The receiver typically acknowledges 1448 per acknowledgement.
- There is no case where the receiver ACKs every other received segment. The receiver ACKs in order and sequentially in this trace.

4	21:43:26.716922	192.168.86.68	128.119.245.12	TCP	1514 55639 → 80 [ACK] Seq=1 Ack=1 Win=131712 Len=1448 TSval=
5	21:43:26.716923	192.168.86.68	128.119.245.12	TCP	1514 55639 → 80 [ACK] Seq=1449 Ack=1 Win=131712 Len=1448 TSv
6	21:43:26.716924	192.168.86.68	128.119.245.12	TCP	1514 55639 → 80 [ACK] Seq=2897 Ack=1 Win=131712 Len=1448 TSv
7	21:43:26.745546	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=1449 Win=31872 Len=0 TSval=:
8	21:43:26.745551	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=2897 Win=34816 Len=0 TSval=:
9	21:43:26.745649	192.168.86.68	128.119.245.12	TCP	1514 55639 → 80 [ACK] Seq=4345 Ack=1 Win=131712 Len=1448 TSv
10	21:43:26.745650	192.168.86.68	128.119.245.12	TCP	1514 55639 → 80 [ACK] Seq=5793 Ack=1 Win=131712 Len=1448 TSv
11	21:43:26.745729	192.168.86.68	128.119.245.12	TCP	1514 55639 → 80 [ACK] Seq=7241 Ack=1 Win=131712 Len=1448 TSv
12	21:43:26.745730	192.168.86.68	128.119.245.12	TCP	1514 55639 → 80 [ACK] Seq=8689 Ack=1 Win=131712 Len=1448 TSv
13	21:43:26.746501	128.119.245.12	192.168.86.68	TCP	66 80 → 55639 [ACK] Seq=1 Ack=4345 Win=37760 Len=0 TSval=:

Question 11: What is the throughput (bytes transferred per unit time) for the TCP connection? Explain how you calculated this value.

- Bits/s A → B: 6667 kbps
- Bits/s B → A: 227 kbps
- $6667 + 227 = 6894$ kbps is the throughput
- I used Wireshark Statistics → Conversations

Address A	Port A	Address B	Port B	Packets	Bytes	Stream ID	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration	Bits/s A → B	Bits/s B → A	Flows
192.168.86.68	55639	128.119.245.12	80	180	166 kB	0	109	161 kB	71	5 kB	0.000000	0.1927	6667 kbps	227 kbps	2

- You can also just add all of bytes sent and divide it over the total elapsed time.